

REF. NO.: EX/EE/PC/B/T/225/2022

SEQUENTIAL SYSTEMS & MICROPROCESSORS

B.E. Electrical Engineering · 2nd Year · 2nd Semester Examination, 2022

TIME: 3 HOURS FULL MARKS: 100 EACH PART: 50

PART I – SEQUENTIAL SYSTEMS

50 marks

Q1

Answer *any Two* of the following (compulsory; $2 \times 5 = 10$ marks)

(A) FICTITIOUS FLIP-FLOP WITH INPUTS A & B

5

The fictitious flip-flop has the following input–output specification:

- $AB = 00 \Rightarrow Q^+ = 0$ (force-reset)
- $AB = 11 \Rightarrow Q^+ = 1$ (force-set)
- $AB = 01 \Rightarrow Q^+ = Q$ (no change / hold)
- $AB = 10 \Rightarrow Q^+ = \bar{Q}$ (toggle / complement)

TRUTH TABLE

A	B	Q (PRESENT)	Q ⁺ (NEXT)	OPERATION
0	0	0	0	Reset
0	0	1	0	
0	1	0	0	Hold
0	1	1	1	
1	0	0	1	Toggle
1	0	1	0	
1	1	0	1	Set
1	1	1	1	

EXCITATION TABLE

For each transition $Q \rightarrow Q^+$, the values of A,B that produce it are read directly from the truth table:

Q	Q ⁺	A	B	JUSTIFICATION
0	0	0	x	AB = 00 (reset) or 01 (hold)
0	1	1	x	AB = 10 (toggle) or 11 (set)
1	0	x	0	AB = 00 (reset) or 10 (toggle)
1	1	x	1	AB = 01 (hold) or 11 (set)

Observation: The excitation table shows that A alone determines the transition $0 \rightarrow \cdot$ and B alone determines $1 \rightarrow \cdot$. Hence the FF has a hybrid set/reset–toggle/hold behaviour.

(B) CONVERSION OF $\overline{R}\overline{S}$ FLIP-FLOP TO RS FLIP-FLOP

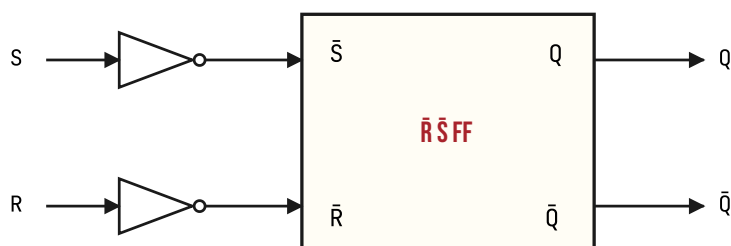
5

The $\overline{R}\overline{S}$ flip-flop (NAND latch) is **active-low**: it sets when $\overline{S} = 0$ and resets when $\overline{R} = 0$. Its truth table is:

$\overline{S}$	$\overline{R}$	OPERATION
0	0	Forbidden / race
0	1	Set ($Q^+ = 1$)
1	0	Reset ($Q^+ = 0$)
1	1	Hold

An ordinary RS FF is **active-high**: it sets on $S = 1$, resets on $R = 1$. To convert $\overline{R}\overline{S}$ to RS we simply place an inverter on each input:

$$\overline{S}_{int} = \overline{S}, \quad \overline{R}_{int} = \overline{R}$$



RS flip-flop synthesised from an active-low $\overline{R}\overline{S}$ latch using two inverters.

Now, when $S = 1$ we get $\overline{S} = 0$ which sets the latch; when $R = 1$ we get $\overline{R} = 0$ which resets it. The forbidden state shifts from $\overline{S} = \overline{R} = 0$ to $S = R = 1$, exactly as in a normal RS FF.

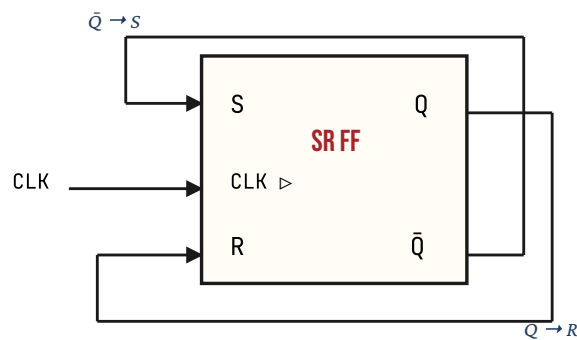
(C) SR FLIP-FLOP IN TOGGLE MODE

5

The SR FF has two restrictions: $SR = 11$ is forbidden, and $SR = 00$ holds. To make it toggle on every clock pulse we must **force** SR **such that** the next state is always $\overline{Q}$.

Required: when $Q = 0$ we want $Q^+ = 1 \rightarrow$ need $S = 1, R = 0$. When $Q = 1$ we want $Q^+ = 0 \rightarrow$ need $S = 0, R = 1$. This is achieved by feeding back:

$$S = \bar{Q}, \quad R = Q$$



SR FF wired as a T flip-flop ($T=1$ always): $S = \bar{Q}, R = Q$.

This connection guarantees $SR \in \{01, 10\}$ — never the forbidden 11, never the hold state 00 — so on each clock edge Q inverts. The circuit therefore behaves as a T flip-flop with $T \equiv 1$, and its output frequency is half the clock frequency (frequency divider).

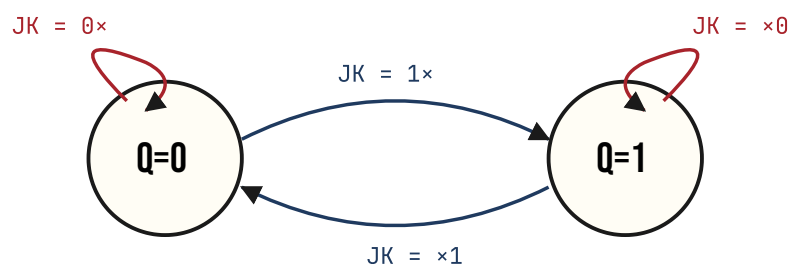
(D) STATE DIAGRAM OF J-K FLIP-FLOP FROM ITS EXCITATION TABLE

5

JK EXCITATION TABLE

Q	Q ⁺	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

The state diagram has only **two states**: $Q = 0$ and $Q = 1$. The labels on the directed arcs are the input combination JK that causes that transition (with don't-cares written as 0/1):



State diagram of JK flip-flop. Self-loops correspond to "hold", crossing arcs to "set"/"reset"/"toggle".

The four behavioural modes of the JK FF are encoded compactly: $JK = 00 \rightarrow \text{hold}$, $01 \rightarrow \text{reset}$, $10 \rightarrow \text{set}$, $11 \rightarrow \text{toggle}$.

Q2

Excitation Table, SR $\rightarrow$ JK Conversion, and Sequential Circuit Analysis

10 + 10

(A) EXCITATION TABLE; CONVERTING SR-FF TO JK-FF

4 + 6

EXCITATION TABLE — DEFINITION

The **excitation table** of a flip-flop lists the values of its inputs required to cause every possible state transition $Q \rightarrow Q^+$. It is the inverse map of the truth table: the truth table answers "given inputs, what is Q^+ ?", whereas the excitation table answers "given desired Q^+ , what inputs are needed?" — making it the natural starting point for synthesis.

SR EXCITATION TABLE

Q	Q ⁺	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0

SR $\rightarrow$ JK CONVERSION TABLE

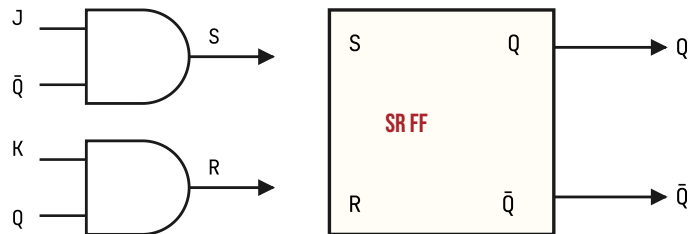
Treat J , K as external inputs and S , R as functions to be derived. For each combination of Q , J , K first read Q^+ from the JK truth table, then look up the required S , R from the SR excitation table:

Q	J	K	Q ⁺	S	R
0	0	0	0	0	x
0	0	1	0	0	x
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	x	0
1	0	1	0	0	1
1	1	0	1	x	0
1	1	1	0	0	1

K-MAP SIMPLIFICATION

From the columns of the table:

$$S = J\bar{Q}, \quad R = KQ$$



SR-to-JK conversion: $S = J\bar{Q}$, $R = KQ$, with Q and $\bar{Q}$ fed back externally.

(B) ANALYSIS OF CIRCUIT IN FIGURE Q2(B) – STATE TABLE & TRANSITION DIAGRAM

8 + 2

The circuit contains two SR flip-flops A and B clocked by the same input CLK . From the figure the gate equations are:

$$S_A = \bar{B}, \quad R_A = B$$

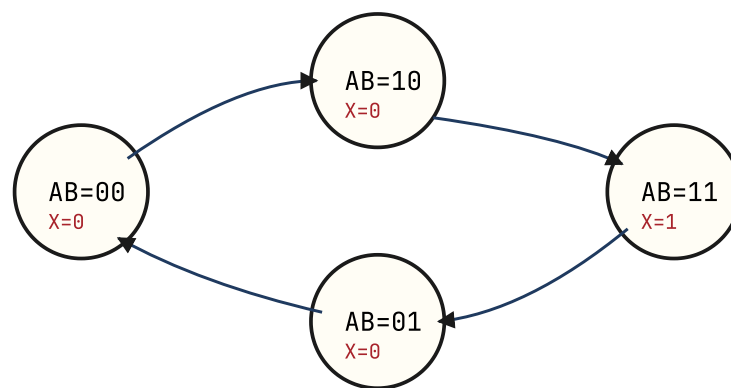
$$S_B = A\bar{B}, \quad R_B = \bar{A}B$$

$$X = AB$$

STATE TABLE

A	B	S_A	R_A	S_B	R_B	A^+	B^+	X
0	0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0	0
1	0	1	0	1	0	1	1	0
1	1	0	1	0	0	0	1	1

STATE TRANSITION DIAGRAM



Mod-4 ring: 00 → 10 → 11 → 01 → 00 ... Output X = 1 once per cycle (when AB=11).

The circuit therefore acts as a **2-bit Gray-code counter** with a single-bit output X that pulses high once every four clock cycles.

Q3

Counters & Shift Registers

10 + 10

(A) SYNCHRONOUS VS ASYNCHRONOUS COUNTERS; DESIGN 3-BIT SYNCHRONOUS UP-COUNTER USING JK FFs

2 + 8

SYNCHRONOUS VS ASYNCHRONOUS

In a **synchronous counter** all flip-flops are driven by a common clock. The next-state inputs are obtained from combinational logic on the present state, so all FFs change simultaneously on the active clock edge. In an **asynchronous (ripple) counter** only the LSB FF is driven by the system clock; each subsequent FF is clocked by the output of the previous stage, producing a propagation delay that ripples through the chain.

3-BIT SYNCHRONOUS UP-COUNTER DESIGN (000 → 001 → ... → 111 → 000)**Step 1 — State table with JK excitations**

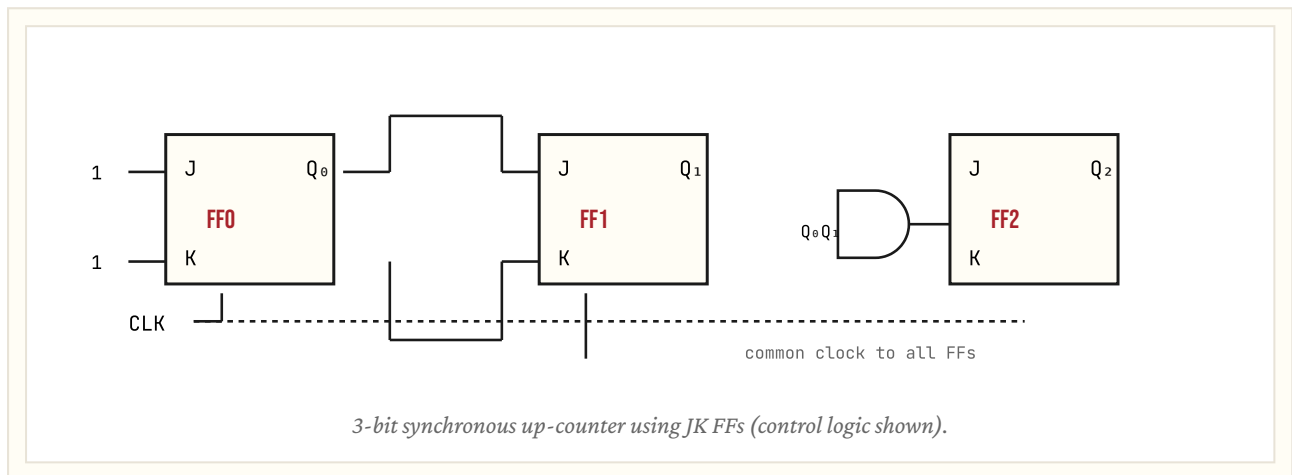
Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	J_2	K_2	J_1	K_1	J_0	K_0
0	0	0	0	0	1	0	x	0	x	1	x
0	0	1	0	1	0	0	x	1	x	x	1
0	1	0	0	1	1	0	x	x	0	1	x
0	1	1	1	0	0	1	x	x	1	x	1
1	0	0	1	0	1	x	0	0	x	1	x
1	0	1	1	1	0	x	0	1	x	x	1
1	1	0	1	1	1	x	0	x	0	1	x
1	1	1	0	0	0	x	1	x	1	x	1

Step 2 — K-map minimisation

$$J_0 = K_0 = 1$$

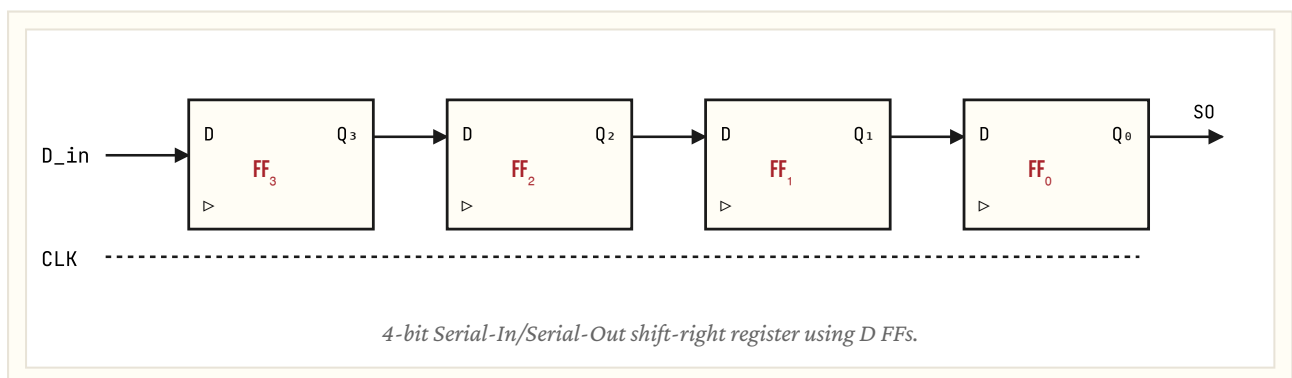
$$J_1 = K_1 = Q_0$$

$$J_2 = K_2 = Q_1Q_0$$

CIRCUIT**(B) 4-BIT SHIFT-RIGHT REGISTER WITH D FFs & SERIAL ENTRY OF "0100"**

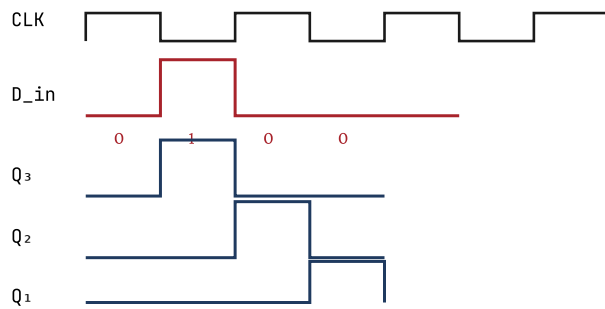
4 + 6

A 4-bit shift-right register is built by cascading four D flip-flops, each clocked by a common clock. The serial input D_{in} drives FF3 and the output of FF_k drives FF_{k-1}:

**SERIAL ENTRY OF "0100" (LSB FIRST, I.E. THE RIGHT-MOST "0" ENTERS FIRST)**

CLOCK	D_{IN}	Q_3	Q_2	Q_1	Q_0
Initial	—	0	0	0	0
1	0	0	0	0	0
2	1	1	0	0	0
3	0	0	1	0	0
4	0	0	0	1	0

After four clock pulses the register contains $Q_3Q_2Q_1Q_0 = 0010$ – the binary "0100" appearing as serially-loaded data. (If MSB-first entry is desired, reverse the input order.)



Timing diagram of "0100" being shifted in serially over 4 clock pulses.

Q4

Mod-3 / Mod-6 counter analysis & consecutive-1s detector

10 + 10

(A) JUSTIFY MOD-3 BEHAVIOUR AND EXTENSION TO MOD-6

4 + 6

For both negative-edge-triggered JK FFs in Figure Q4(a):

$$J_A = \bar{B}, \quad K_A = 1; \quad J_B = A, \quad K_B = 1$$

PRESENT A, B	J_A, K_A	A^+	J_B, K_B	B^+
0 0	1, 1	1	0, 1	0
1 0	1, 1	0	1, 1	1
0 1	0, 1	0	0, 1	0
1 1	0, 1	0	1, 1	0

The reachable cycle starting from 00 is $00 \rightarrow 10 \rightarrow 01 \rightarrow 00$, three states only — hence **Mod-3**. The unused state 11 self-corrects to 00 on the next clock.

EXTENSION TO MOD-6

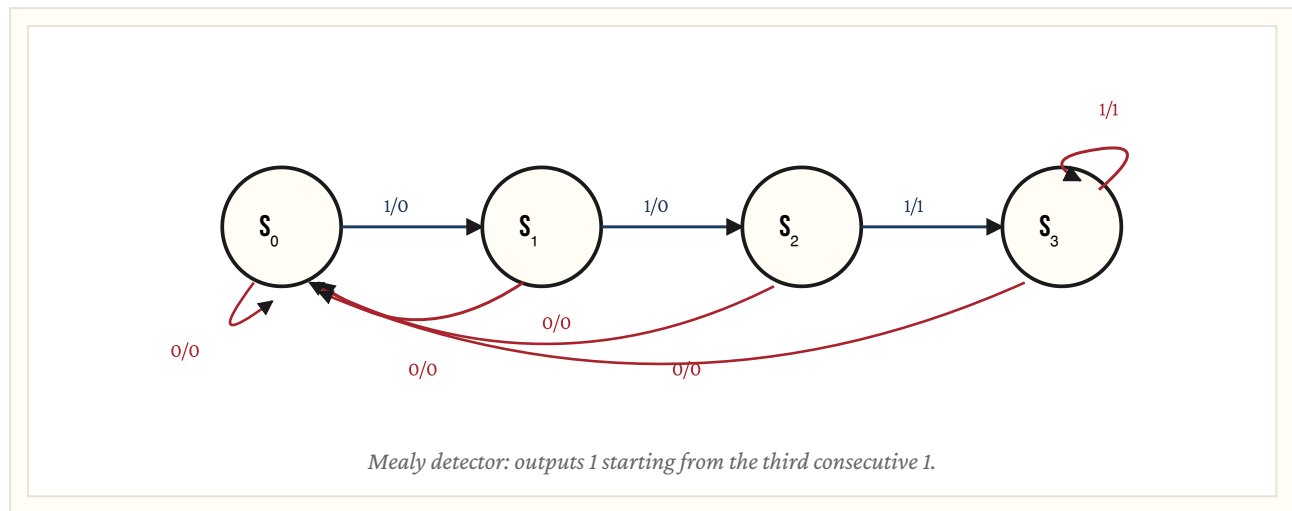
Connecting B to the clock input of a third JK FF C with $J_C = K_C = 1$ (toggle on every B-falling-edge) gives C a period twice that of B. Since the inner stage has period 3, the cascaded system has period $3 \times 2 = 6$ — a **Mod-6 counter**:

N	C	B	A	N	C	B	A
0	0	0	0	3	1	0	0
1	0	0	1	4	1	0	1
2	0	1	0	5	1	1	0

(B) SEQUENCE DETECTOR — THREE OR MORE CONSECUTIVE 1S (MEALY)

4 + 6

STATE TRANSITION DIAGRAM



STATE TABLE & D-FF EXCITATION

Encode: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$. Let present state = Q_1Q_0 , input = x , output = z .

Q_1	Q_0	X	$Q_1^+ = D_1$	$Q_0^+ = D_0$	Z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	1	1	1

$$D_1 = x(Q_0 + Q_1), \quad D_0 = x(\bar{Q}_1\bar{Q}_0 + Q_1), \quad z = xQ_1$$

Q5

Synchronous circuit synthesis & Universal Shift Register

10 + 10

[A] SYNC. CIRCUIT SYNTHESIS FROM GIVEN STD USING JK FFS

10

The given diagram shows four states $\{00, 01, 10, 11\}$ with input $X \in \{0, 1\}$. Reading the arcs (state/input $\rightarrow$ next state):

Q_1	Q_0	$X=0$	$X=1$
00		00	01
01		01	11
11		11	00

10	10	11
----	----	----

Replacing each present-state/next-state pair with the JK excitation requirements:

Q_1	Q_0	X	Q_1^+	Q_0^+	J_1	K_1	J_0	K_0
0	0	0	0	0	0	x	0	x
0	0	1	0	1	0	x	1	x
0	1	0	0	1	0	x	x	0
0	1	1	1	1	1	x	x	0
1	0	0	1	0	x	0	0	x
1	0	1	1	1	x	0	1	x
1	1	0	1	1	x	0	x	0
1	1	1	0	0	x	1	x	1

K-map reduction yields:

$$J_1 = XQ_0, \quad K_1 = XQ_0$$

$$J_0 = X, \quad K_0 = XQ_1$$

Each FF is wired with its J and K driven by these AND gates of inputs/outputs.

(B) UNIVERSAL SHIFT REGISTER – DEFINITION AND 4-BIT MODES

2 + 8

A **universal shift register** is a register that can perform all four standard register operations: parallel load, shift right, shift left, and hold (no change). It is constructed from D flip-flops preceded by 4-to-1 multiplexers driven by 2 mode-select lines s_1s_0 :

s_1	s_0	OPERATION	MUX SELECTS
0	0	Hold (no change)	Q_i (own output)
0	1	Shift right	Q_{i+1} (right-neighbour)
1	0	Shift left	Q_{i-1} (left-neighbour)
1	1	Parallel load	External I_i

4-BIT UNIVERSAL SHIFT REGISTER (74194 STYLE)



Each stage = 4:1 MUX → D-FF, common s_1s_0 , common CLK

Architecture: 4 D-FFs each preceded by 4:1 MUX selecting hold/right/left/load input.

PART II – 8085 MICROPROCESSOR

50 marks

Q1

Address space, PC/SP, and 8085 pin functions

5 + 5

(A) MEMORY LOCATIONS ADDRESSABLE WITH 14 LINES; PC AND SP FUNCTIONS

2 + 3

With n address pins a microprocessor can directly access 2^n distinct memory locations. For $n = 14$:

$$N = 2^{14} = 16,384 \text{ locations} = 16 \text{ K}$$

Program Counter (PC): a 16-bit register that holds the address of the *next instruction* to be fetched. After every opcode fetch the PC is auto-incremented; on jumps/calls/returns it is loaded with the new target.

Stack Pointer (SP): a 16-bit register that points to the top of the LIFO stack maintained in RAM. PUSH instructions decrement SP by 2; POP, RET and CALL adjust SP automatically. SP is used for return-address storage, register save/restore, and parameter passing in subroutines.

(B) FUNCTIONS OF PINS S_0 , S_1 , $IO/\bar{M}$

3 + 2

S_0 , S_1 (Status pins): together they encode the type of machine cycle currently being executed:

S_1	S_0	MACHINE CYCLE
0	0	HALT
0	1	Memory / I-O Write
1	0	Memory / I-O Read
1	1	Opcode Fetch

$IO/\bar{M}$: a single status line that distinguishes the address space being accessed. $IO/\bar{M} = 0 \Rightarrow$ memory operation; $IO/\bar{M} = 1 \Rightarrow$ I/O operation. Combined with $\overline{RD} / \overline{WR}$ it generates four distinct control strobes: $\overline{MEMR}$, $\overline{MEMW}$, $\overline{IOR}$, $\overline{IOW}$.

Q2

Addressing modes & address-bus multiplexing

5 + 5

(A) ADDRESSING MODES OF GIVEN INSTRUCTIONS

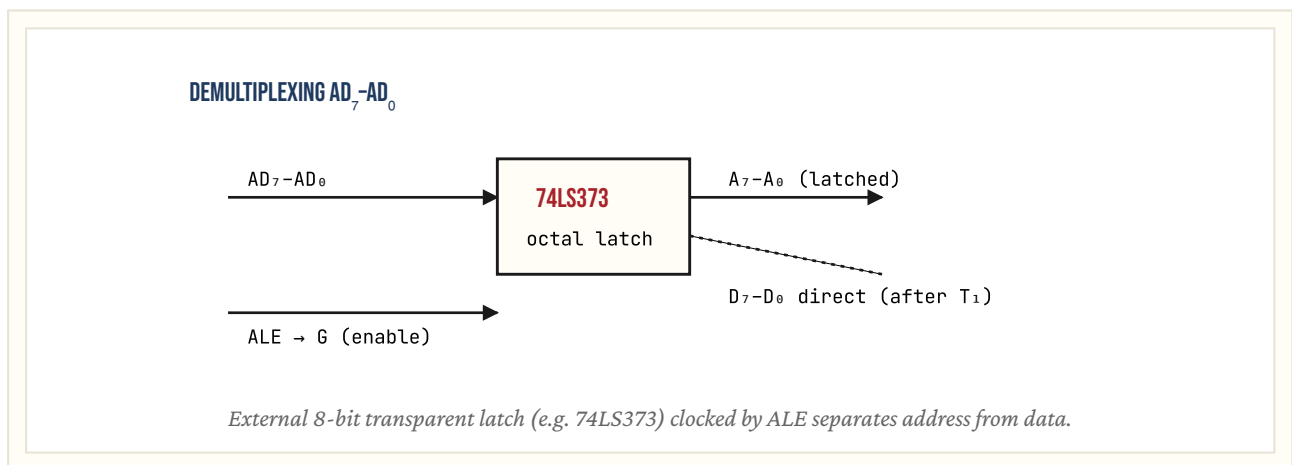
5

INSTRUCTION	ADDRESSING MODE	REASON
CMC	Implied / Implicit	Operand (CY flag) is fixed by the opcode
ORA B	Register	Operand B is named in the opcode
SUI 05H	Immediate	05H is supplied as the second byte of the instruction
MOV B, M	Register-indirect	M $\equiv$ memory location pointed to by HL pair
STA 8000H	Direct	Address 8000H is given explicitly

(B) TIME-MULTIPLEXING OF LOWER ADDRESS BUS WITH DATA BUS

5

The 8085 has 40 pins; to keep the package small the lower-order address bits A_7-A_0 are **time-multiplexed** with the data bus D_7-D_0 on pins AD_7-AD_0 . During the first T-state of every machine cycle the AD lines carry the lower address; thereafter they carry data. The CPU asserts the **ALE** (Address Latch Enable) signal during T_1 to inform external logic that valid address is present.

**Q3****Opcode-Fetch vs Memory R/W; Wait-state generation**

5 + 5

(A) OPCODE-FETCH VS MEMORY-READ/WRITE TIMING

5

FEATURE	OPCODE FETCH	MEMORY READ/WRITE
Status (S_1 S_0)	1 1	1 0 (read), 0 1 (write)
T-states	4 (or 6 for some)	3
$I\bar{O}/\bar{M}$	0	0

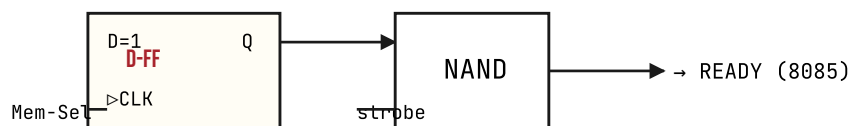
Source of address	PC	HL/BC/DE or 16-bit op
Action in T_4	Decode + execute	—

The opcode fetch is unique because it includes T_4 (and possibly T_5, T_6) reserved for instruction decoding inside the CPU; during these extra T-states the address/data lines are idle externally.

(B) WAIT-STATE GENERATION CIRCUIT

5

Slow memory/IO devices cannot complete a transfer in the standard 3 T-states. Pulling the **READY** input low at the end of T_2 makes the CPU insert wait states (T_W) until READY is sampled high again. A typical one-wait-state generator uses a D-flip-flop clocked by the CPU clock and a memory-select strobe:



One-wait-state circuit: a D-FF samples device-select at clock; output gates READY low for one T cycle.

Q4

Accumulator and flag register after each instruction

10

INITIAL A = 7FH; FLAGS RESET. TRACE EACH INSTRUCTION INDEPENDENTLY.

10

Flag bit positions in PSW: $S Z X AC X P X CY$ (bits 7,6,4,2,0).

#	INSTRUCTION	OPERATION	A AFTER	S	Z	AC	P	CY
i	DCR A	$7F - 01 = 7E$	7EH	0	0	0	1	— *
ii	MOV B, A	copy A → B (A unchanged, no flags)	7FH	0	0	0	0	0
iii	CPI 90H	$7F - 90 = EF$ (with borrow)	7FH	1	0	1	1	1
iv	ADI 92H	$7F + 92 = 111$ (carry out)	11H	0	0	1	1	1
v	RAL	shift left through CY: $7F \rightarrow FE$, CY=0	FEH	1	0	0	0	0

*DCR does not affect CY.

Q5

Function and execution steps of CALL (or RST)

10

CALL ADDR16 – FUNCTION AND STEP-BY-STEP EXECUTION

10

CALL transfers control to a subroutine while saving the current PC on the stack so that RET can return. It is a 3-byte, 18 T-states (5-machine-cycle) instruction.

1. **OF (4 T):** CPU fetches opcode CD from PC, decodes it.
2. **MR (3 T):** Reads low byte of target address.
3. **MR (3 T):** Reads high byte → forms 16-bit target.
4. **MW (3 T):** $SP \leftarrow SP - 1$; pushes high byte of (incremented) PC onto stack.
5. **MW (3 T):** $SP \leftarrow SP - 1$; pushes low byte; $PC \leftarrow$ target address.

RET reverses the last two steps: pops the saved address from the stack into PC.

Q6

Delay-loop calculation and SIM masking

6 + 4

(A) SYSTEM CLOCK AND DELAY CALCULATION FOR THE GIVEN SUBROUTINE

2 + 4

The 8085 internally divides the crystal frequency by 2:

$$f_{sys} = \frac{4 \text{ MHz}}{2} = 2 \text{ MHz} \Rightarrow T = 0.5 \mu s$$

Inner loop body (DCR C, JNZ Loop1) executes $(4 + 10) = 14$ T-states for 254 iterations and $(4 + 7) = 11$ T-states on the last iteration:

$$T_{inner} = 254 \times 14 + 11 = 3567 \text{ T-states}$$

One pass of the outer loop = MVI C, FFH (7) + T_{inner} + DCR B (4) + JNZ Loop2 (10 or 7):

$$T_{outer/iter} = 7 + 3567 + 4 + 10 = 3588 \text{ T-states (last: 3585)}$$

Outer iterates B = 38H = 56 times:

$$T_{total} \approx 7 (\text{MVI B, 38H}) + 55 \times 3588 + 3585 = 7 + 197340 + 3585 = 200,932 \text{ T-states}$$

$$\Delta t = 200,932 \times 0.5 \mu s \approx 100.5 \text{ ms}$$

(B) USE OF SIM TO MASK INTERRUPTS

4

SIM (Set Interrupt Mask) writes the lower 5 bits of A into the interrupt mask register:

BIT	NAME	FUNCTION
D ₀	M5.5	Mask RST 5.5 (1 = mask)
D ₁	M6.5	Mask RST 6.5
D ₂	M7.5	Mask RST 7.5
D ₃	MSE	1 = enable above mask bits
D ₄	R7.5	Reset RST 7.5 flip-flop

Example — to mask RST 6.5 only: **MVI A, 0AH ; SIM** (binary 00001010 ⇒ MSE=1, M6.5=1).

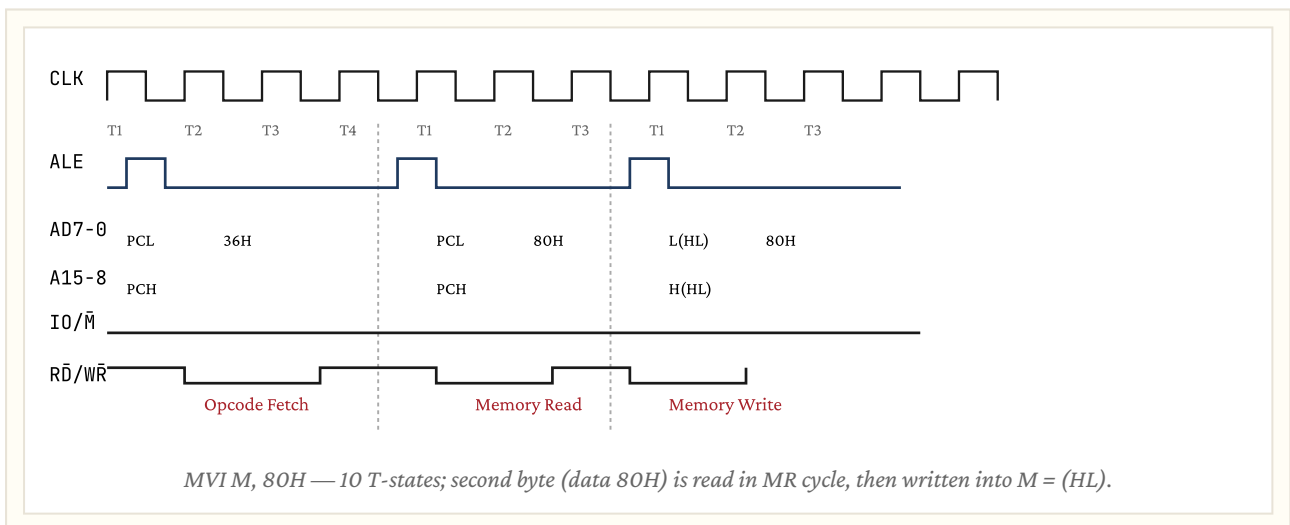
Q7**Timing diagram of MVI M, 80H & maskable/non-maskable interrupts**

7 + 3

(A) MVI M, 80H — TIMING DIAGRAM (OPCODE 36H)

7

The instruction is 2 bytes, 3 machine cycles (OF + MR + MW), 10 T-states.

**(B) MASKABLE VS NON-MASKABLE INTERRUPTS**

3

Maskable interrupts can be enabled or disabled by software (EI, DI, SIM). In 8085 these are INTR, RST 5.5, RST 6.5, RST 7.5. **Non-maskable interrupts** cannot be turned off — TRAP is the only NMI in 8085, used for catastrophic events (power failure, watchdog) and has the highest priority.

(I) CLEAR THE FLAG REGISTER

4

The PSW is loaded by POP PSW. Push 0000H so that the popped PSW = 00:

```
PUSH H          ; save HL temporarily
LXI H, 0000H    ; clear HL
PUSH H          ; 0000H on top of stack (PSW)
POP PSW         ; A = 00, all flags = 0
POP H           ; restore HL
HLT
```

(II) MULTIPLY TWO 8-BIT NUMBERS (REPEATED ADDITION)

6

```
          ; Multiplicand at 8000H, multiplier at 8001H, product (16-bit) at 8050H-805
LDA 8000H
MOV B, A      ; B = multiplicand
LDA 8001H
MOV C, A      ; C = multiplier (loop counter)
LXI H, 0000H  ; HL accumulates 16-bit product
MVI D, 00H    ; DE will hold sign-extended B
MOV E, B

LOOP:
MOV A, C
CPI 00H
JZ  DONE
DAD D          ; HL = HL + DE
DCR C
JMP LOOP

DONE:
SHLD 8050H
HLT
```

— END OF 2022 SOLUTIONS —